File Name: data_preprocessing_visualization.py

Perform data preprocessing, aggregation, and visualization using Pandas, Seaborn, and Matplotlib.

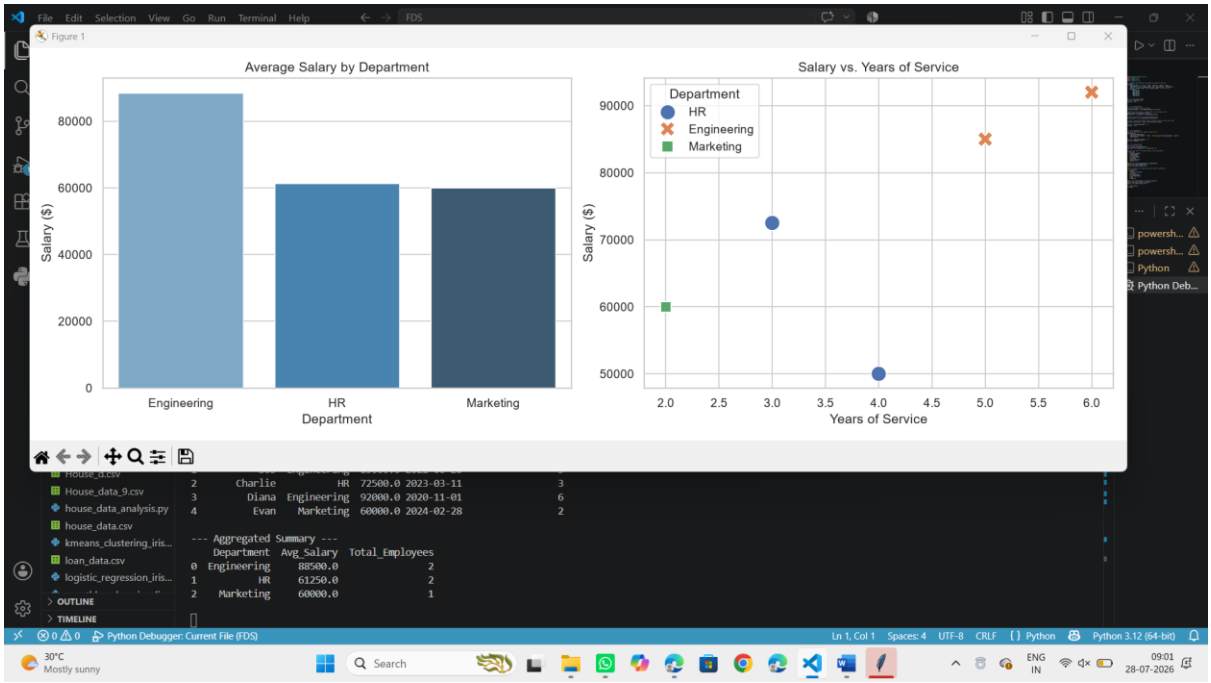
CODE:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# 1. GENERATE RAW MOCK DATA (Simulating an imperfect CSV file)
raw_data = {
    "Employee_Name": [" Alice ", "Bob", "Charlie", "Diana", "Evan"],
    "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
    "Salary": [50000, 85000, np.nan, 92000, 60000],  # Missing value
    "Join_Date": [
        "2022-01-15",
        "2021-06-20",
        "2023-03-11",
        "2020-11-01",
        "2024-02-28",
    ],
}
df = pd.DataFrame(raw_data)
print("--- Raw Data ---")
print(df, "\n")



# 2. DATA PREPROCESSING
# Clean whitespace from text strings
df["Employee_Name"] = df["Employee_Name"].str.strip()

# Handle missing values by replacing NaN with the median salary
median_salary = df["Salary"].median()
df["Salary"] = df["Salary"].fillna(median_salary)

# Convert string dates into proper datetime format
df["Join_Date"] = pd.to_datetime(df["Join_Date"])

# Create a calculated feature (Years of service based on current year 2026)
df["Years_of_Service"] = 2026 - df["Join_Date"].dt.year

print("--- Preprocessed Data ---")
print(df, "\n")
```

```
# 3. DATA AGGREGATION
# Group by department and compute average metrics
dept_summary = (
    df.groupby("Department")
    .agg(Avg_Salary=("Salary", "mean"), Total_Employees=("Employee_Name", "count"))
    .reset_index()
)
print("--- Aggregated Summary ---")
print(dept_summary, "\n")


# 4. DATA VISUALIZATION
# Set visual style theme
sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Bar chart of average salaries across departments
sns.barplot(
    data=dept_summary,
    x="Department",
    y="Avg_Salary",
    hue="Department",
    ax=axes[0],
    palette="Blues_d",
    legend=False,
)
axes[0].set_title("Average Salary by Department")
axes[0].set_xlabel("Department")
axes[0].set_ylabel("Salary ($)")

# Plot 2: Scatter plot mapping experience against compensation
sns.scatterplot(
    data=df,
    x="Years_of_Service",
    y="Salary",
    hue="Department",
    style="Department",
    s=200,
    ax=axes[1],
)
axes[1].set_title("Salary vs. Years of Service")
axes[1].set_xlabel("Years of Service")
axes[1].set_ylabel("Salary ($)")

plt.tight_layout()
plt.show()
```

Output:

```
PS D:\FDS>  d:; cd 'd:\FDS'; & 'C:\Program Files\Python312\python.exe' 'c:\Users\ACER\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '62383' '--' 'D:\FDS\data_preprocessing_visualization.py'
--- Raw Data ---
  Employee_Name   Department   Salary   Join_Date
0         Alice           HR  50000.0  2022-01-15
1           Bob  Engineering  85000.0  2021-06-20
2       Charlie           HR      NaN  2023-03-11
3         Diana  Engineering  92000.0  2020-11-01
4          Evan    Marketing  60000.0  2024-02-28

--- Preprocessed Data ---
  Employee_Name   Department   Salary  Join_Date  Years_of_Service
0         Alice           HR  50000.0 2022-01-15                 4
1           Bob  Engineering  85000.0 2021-06-20                 5
2       Charlie           HR  72500.0 2023-03-11                 3
3         Diana  Engineering  92000.0 2020-11-01                 6
4          Evan    Marketing  60000.0 2024-02-28                 2

--- Aggregated Summary ---
    Department  Avg_Salary  Total_Employees
0  Engineering     88500.0                2
1           HR     61250.0                2
2    Marketing     60000.0                1
```

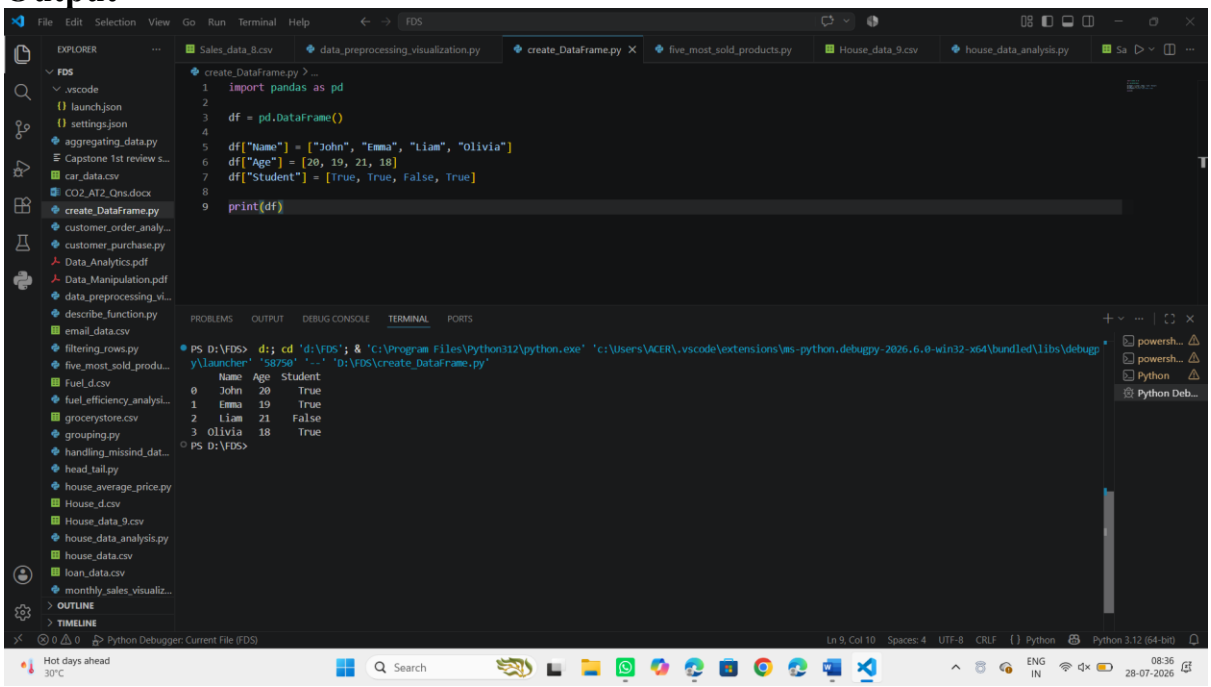


Creating Sample DataFrame

A DataFrame is the core data structure in pandas, used to store data in a tabular form. In this section, we create a DataFrame manually by assigning values to columns, which is useful for small datasets or testing.

```
import pandas as pd
df = pd.DataFrame()

df['Name'] = ['John', 'Emma', 'Liam', 'Olivia']
df['Age'] = [20, 19, 21, 18]
df['Student'] = [True, True, False, True]

print(df)
```

Output

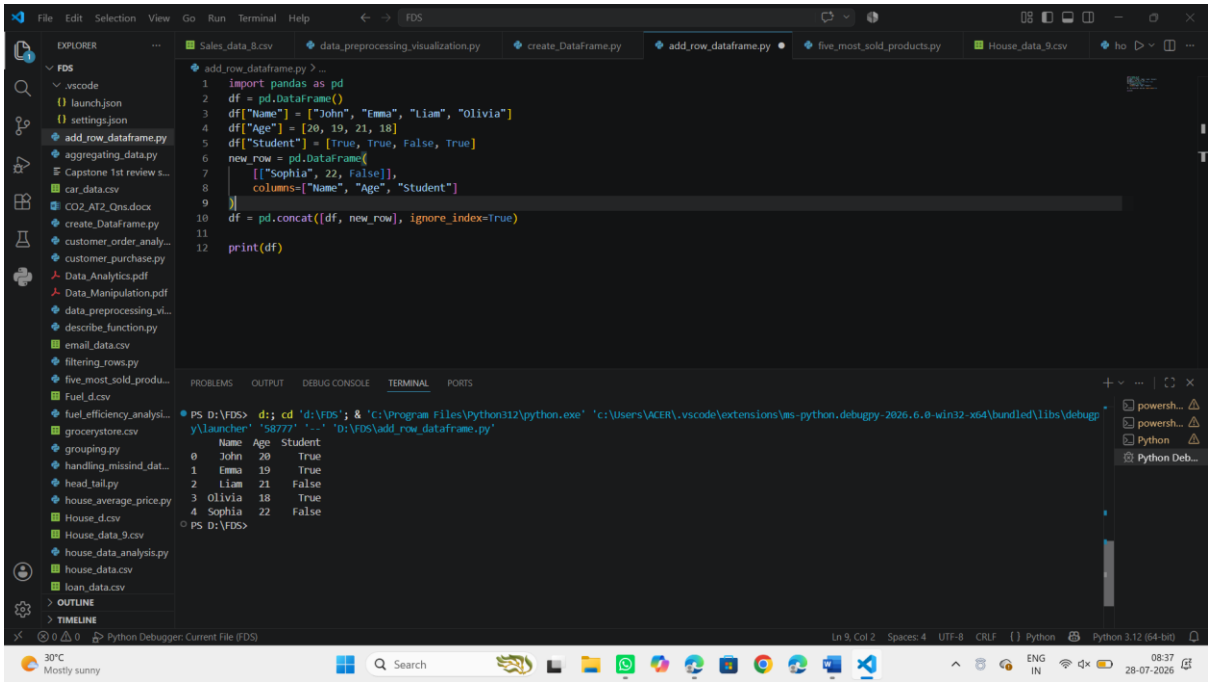


Adding Row to DataFrame

To add new records to an existing DataFrame, Pandas provides the pd.concat() function. It combines two DataFrames together, making it the correct and recommended way to add rows in modern Pandas versions.

```
new_row = pd.DataFrame([['Sophia', 22, False]], columns=['Name', 'Age', 'Student'])
df = pd.concat([df, new_row], ignore_index=True)
print(df)
```

Output



Data Analysis with SciPy

SciPy (Scientific Python) is an open-source Python library for scientific computing and data analysis. Built on top of NumPy, it provides tools for statistics, optimization, signal processing and other mathematical operations.

- Provides statistical and mathematical functions.
- Supports optimization and signal processing.
- Widely used in data analysis, machine learning and research.

1. Importing Required Libraries

- Import SciPy and NumPy libraries.
- NumPy arrays are commonly used with SciPy functions.

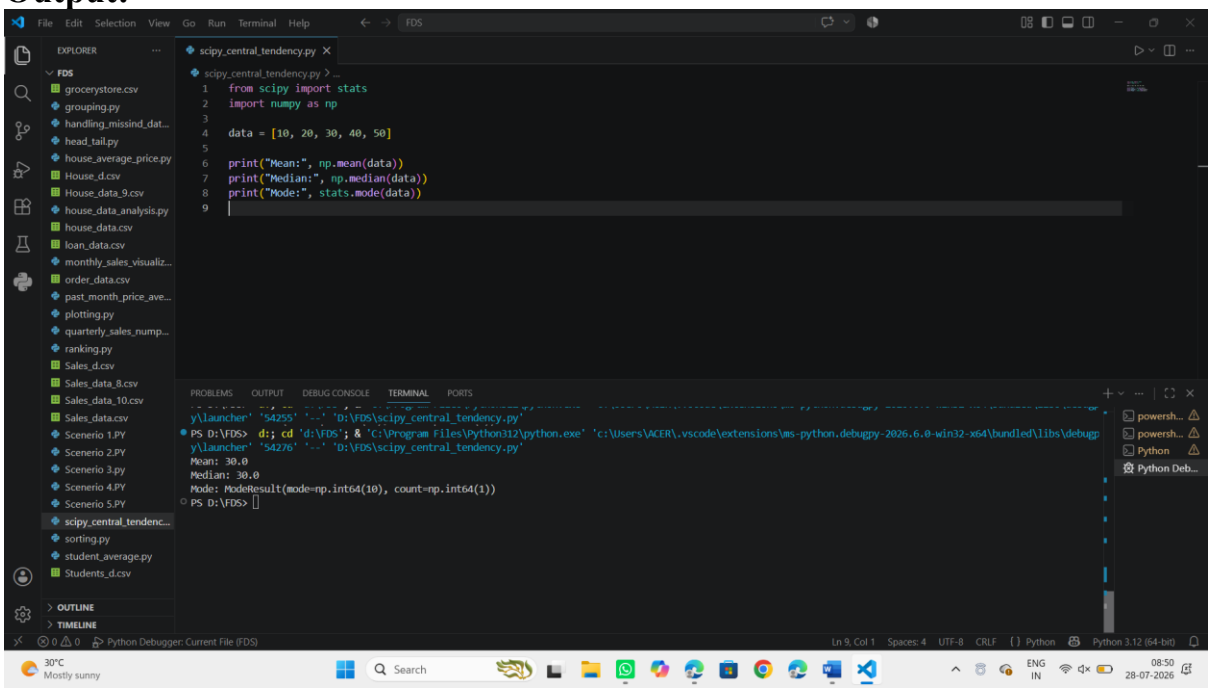
```
import numpy as np
import pandas as pd
```

2. Measures of Central Tendency Using SciPy

- Mean represents the average value.
- Median represents the middle value.
- Mode represents the most frequent value.

```
from scipy import stats
import numpy as np

data = [10, 20, 30, 40, 50]

print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Mode:", stats.mode(data))
```

Output:



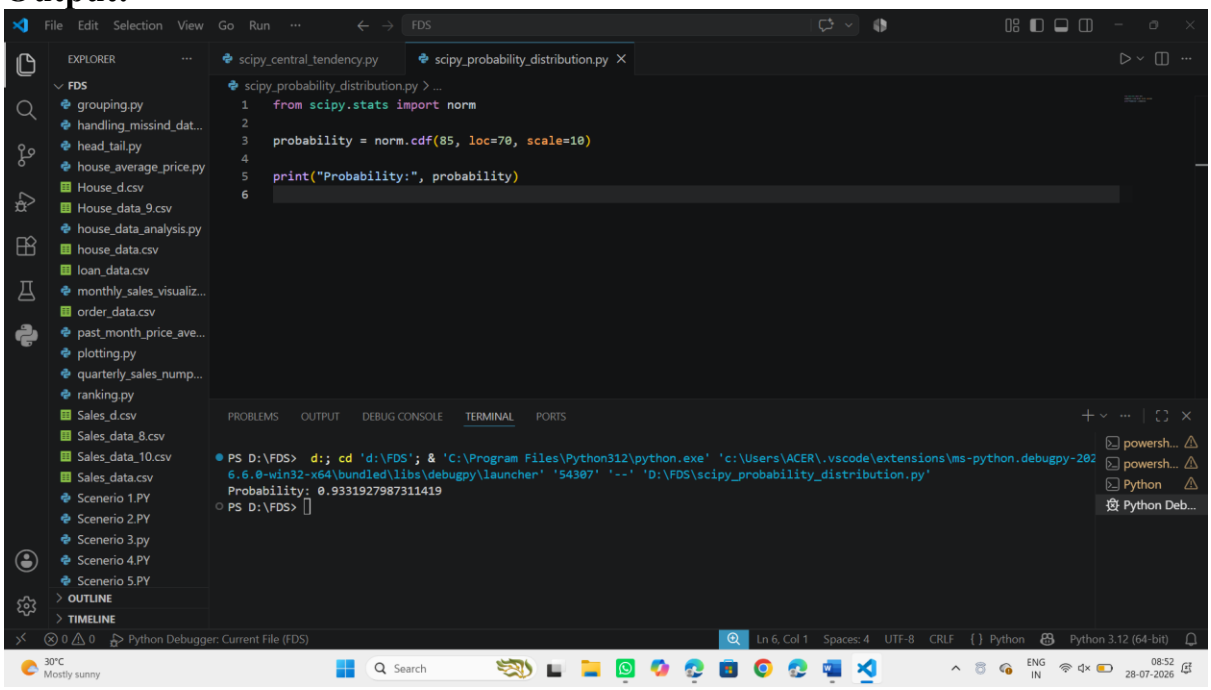
3. Probability Distribution Analysis Using SciPy

Probability distributions describe how data values are distributed.

- loc=70 specifies the mean of the distribution.
- scale=10 specifies the standard deviation.
- norm.cdf() calculates the cumulative probability for a given value.
- The result represents the probability of obtaining a value less than or equal to the specified value (85 in this example).

```
from scipy.stats import norm

probability = norm.cdf(85, loc=70, scale=10)

print("Probability:", probability)
```

Output:



4. Hypothesis Testing

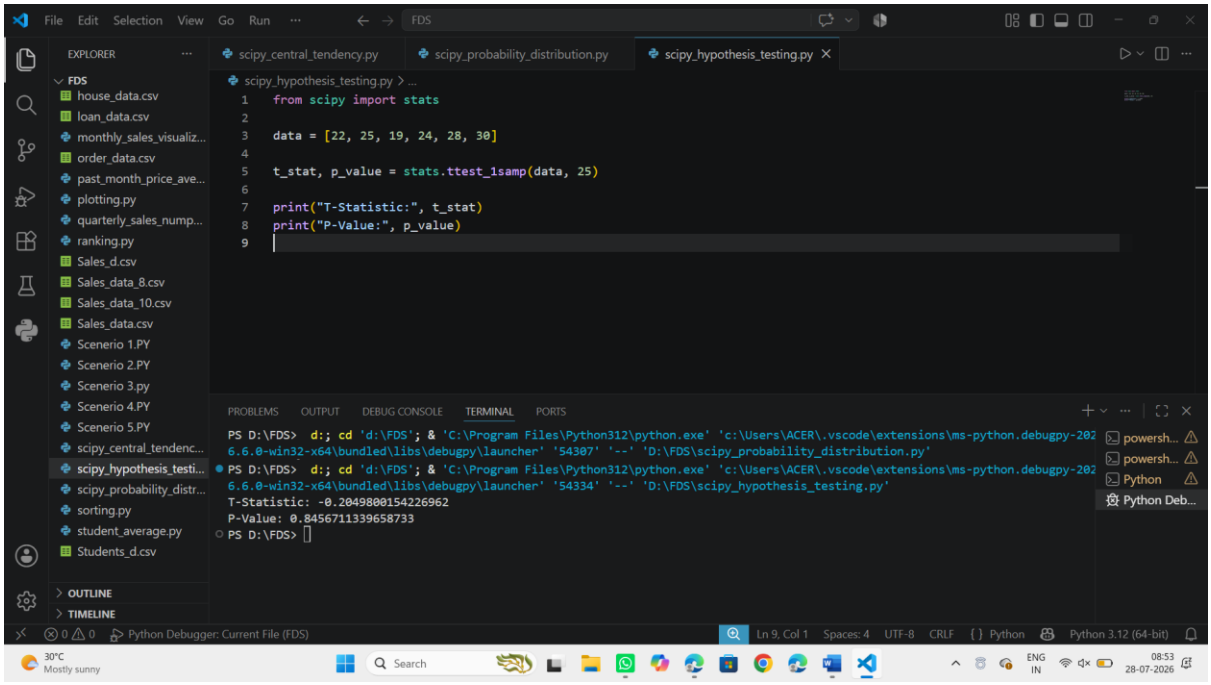
Hypothesis testing helps determine whether a statistical claim is supported by data. SciPy provides functions for t-tests, chi-square tests and other statistical tests.

- Tests whether the sample mean differs from a given value.
- A small p-value indicates a statistically significant difference.

```
from scipy import stats

data = [22, 25, 19, 24, 28, 30]

t_stat, p_value = stats.ttest_1samp(data, 25)

print("T-Statistic:", t_stat)
```

```
print("P-Value:", p_value)
```

Output:



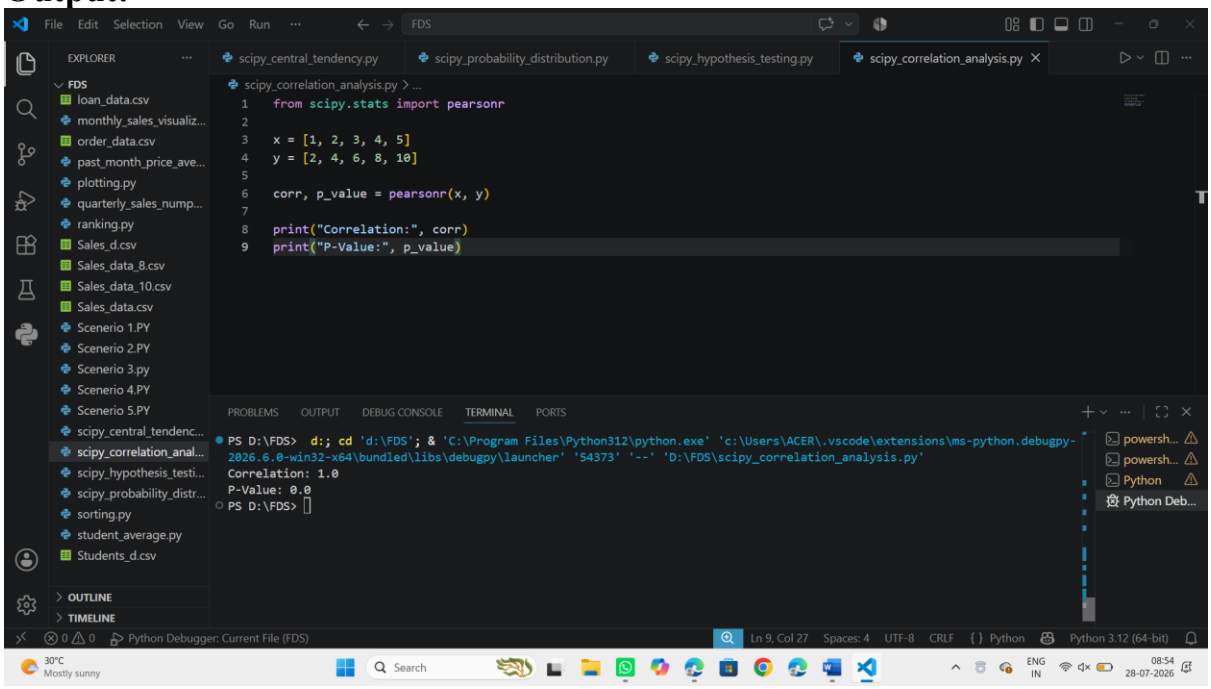
5. Correlation Analysis

Correlation measures the strength and direction of the relationship between two variables.

- Pearson correlation ranges from -1 to 1.
- Values close to 1 indicate a strong positive relationship.

```
from scipy.stats import pearsonr

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

corr, p_value = pearsonr(x, y)

print("Correlation:", corr)
```

Output:



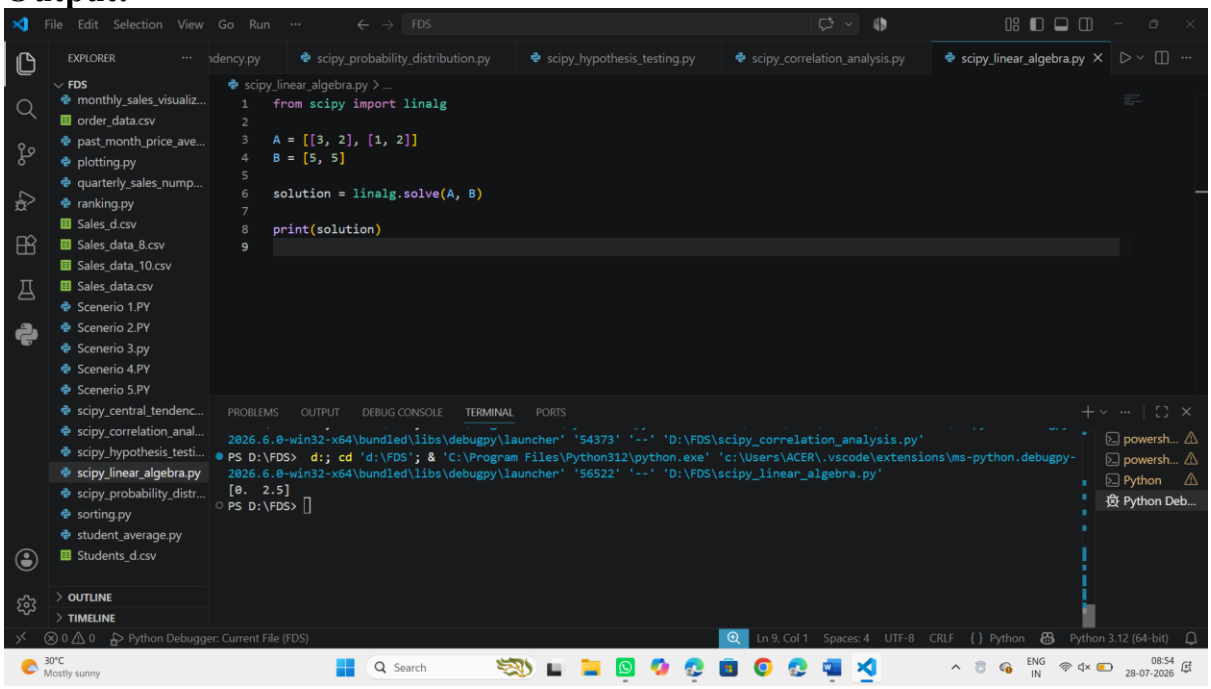
6. Linear Algebra Operations

SciPy provides efficient functions for matrix operations and solving linear equations.

- A represents the coefficients of the linear equations.
- B represents the constant values on the right hand side of the equations.
- linalg.solve() computes the values of the unknown variables that satisfy the equations.

```
from scipy import linalg

A = [[3, 2], [1, 2]]
B = [5, 5]

solution = linalg.solve(A, B)

print(solution)
```

Output:



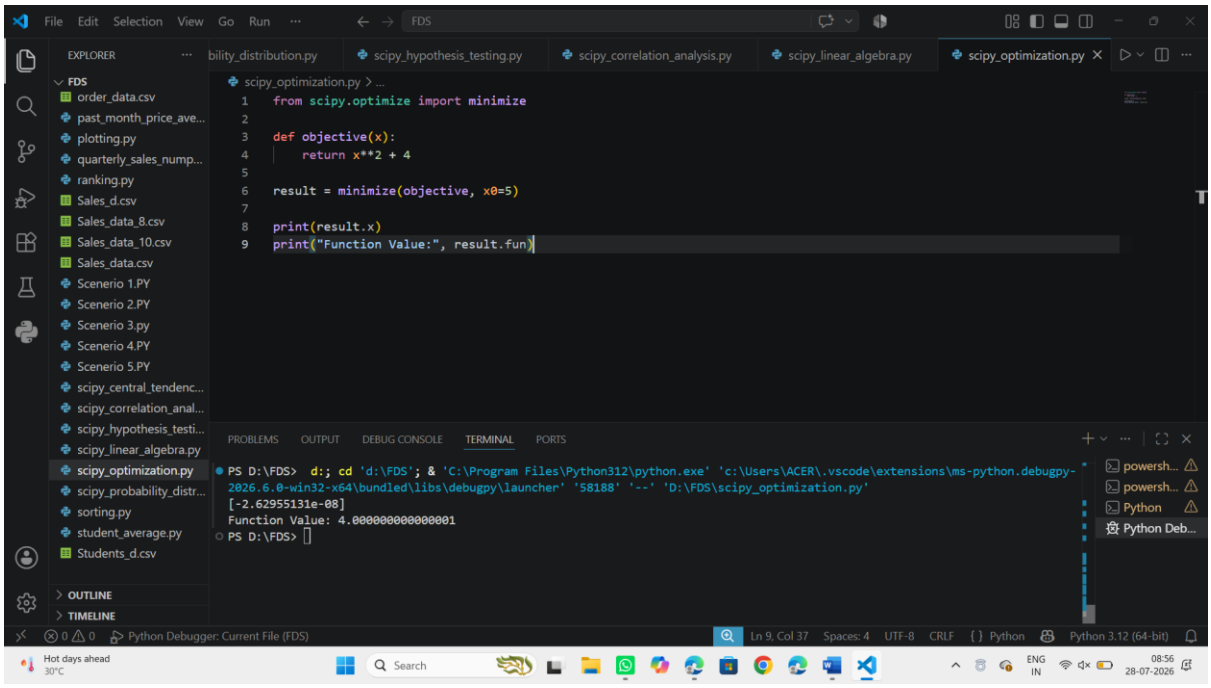
7. Optimization Using SciPy

Optimization is used to find the best solution to a problem by minimizing or maximizing a function.

- objective() defines the function to optimize.
- minimize() finds the value of x that minimizes the function.
- x0=5 specifies the starting point for the search.
- The result returns the optimal value of x.

```
from scipy.optimize import minimize

def objective(x):
    return x**2 + 4

result = minimize(objective, x0=5)

print(result.x)
```

Output:


File Edit Selection View Go Run
FDS
EXPLORER
scipy_optimization.py
scipy_optimization.py > ...
1 from scipy.optimize import minimize
2
3 def objective(x):
4 return x**2 + 4
5
6 result = minimize(objective, x0=5)
7
8 print(result.x)
9 print("Function Value:", result.fun)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\FDS> d:; cd 'd:\FDS'; & 'C:\Program Files\Python312\python.exe' 'c:\Users\ACER\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '58188' '--' 'D:\FDS\scipy_optimization.py'
[-2.62955131e-08]
Function Value: 4.000000000000001
PS D:\FDS>
OUTLINE
TIMELINE
Python Debugger: Current File (FDS)
Ln 9, Col 37 Spaces: 4 UTF-8 CRLF { } Python Python 3.12 (64-bit)
Hot days ahead
30°C
Search
ENG
IN
08:56
28-07-2026

Scikit Library in Python

Scikit-learn (also written as sklearn) is a popular open-source Python library for machine learning. Built on top of NumPy, SciPy and Matplotlib, it provides efficient tools for data analysis and predictive modeling. It offers simple and reusable functions to build models for tasks such as classification, regression, clustering and dimensionality reduction.

Machine Learning Techniques Supported by Scikit-learn

1. Supervised Learning

Supervised learning involves training models using labeled data, where the correct output is already known.

- **Classification:** Scikit-learn provides multiple algorithms to predict categorical outcomes, such as logistic regression, decision trees, random forests, support vector machines (SVMs) and gradient boosting.
- **Regression:** Regression models are used to predict continuous numerical values. Scikit-learn supports linear regression, support vector regression and decision tree regression.

Example: Logistic Regression Algorithm

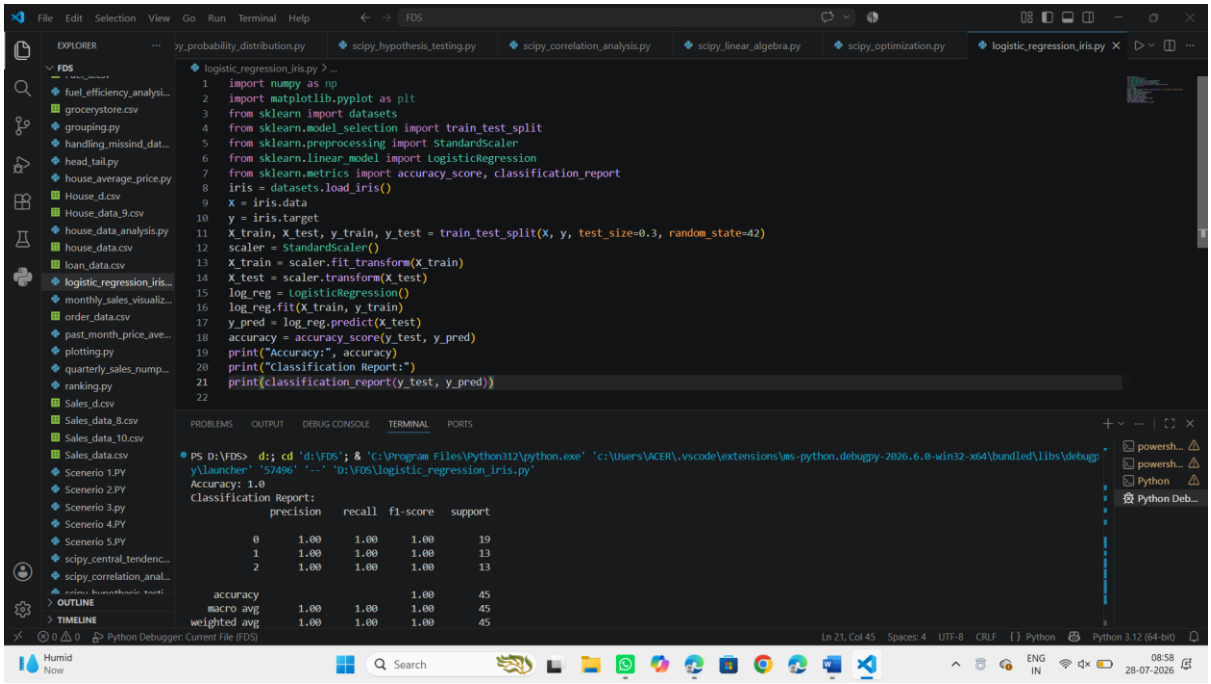
Logistic Regression is a supervised machine learning algorithm used to predict categories (yes/no, spam/not spam, disease/no disease). It works by estimating the probability of an outcome and is simple, easy to interpret and effective for problems where classes can be separated.

This example uses Logistic Regression to classify flowers in the Iris dataset and check how accurately the model predicts their types.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

iris = datasets.load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)

y_pred = log_reg.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Output



2. Unsupervised Learning

Unsupervised learning works with unlabeled data to discover patterns and structure.

- **Clustering:** Scikit-learn offers clustering techniques to group similar data points, including K-means clustering, DBSCAN and hierarchical clustering.
- **Dimensionality Reduction:** To handle high-dimensional data efficiently, Scikit-learn provides techniques like principal component analysis (PCA).

Example: KMeans Algorithm

KMeans groups data into k clusters based on similarity. It is an unsupervised learning algorithm, ideal for tasks like customer segmentation, image compression and anomaly detection, especially when the underlying data structure is unknown.

This program demonstrates how to use KMeans clustering from Scikit-learn to group the Iris dataset into three clusters based on feature similarity.

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

iris = load_iris()
kmeans = KMeans(n_clusters=3)

kmeans.fit(iris.data)
cluster_labels = kmeans.labels_

print("Cluster Labels:", cluster_labels)
```

Output